

C# & .NET Framework

Interview / Exam Question Bank with Explanations, Code Examples & Diagrams

15 Questions | Concepts, OOP, Syntax & Language Features

1 What is Managed Code? Explain Managed and Unmanaged Code

Managed code is code that is executed and controlled by the Common Language Runtime (CLR) of the .NET Framework. The CLR manages memory, security, exception handling, and other runtime services.

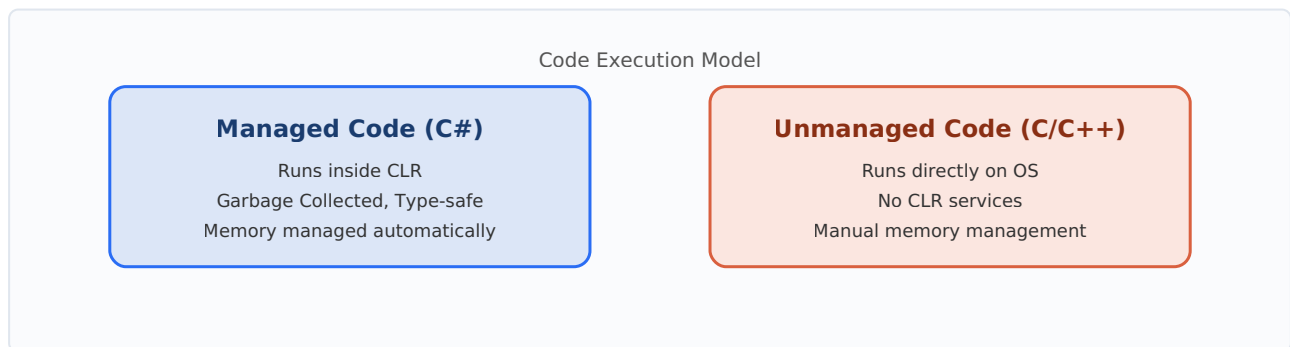
Managed Code

- Executes under the control of CLR.
- Automatic memory management through Garbage Collector (GC).
- Provides type safety and exception handling.
- Example: C# code executed by .NET CLR.

Unmanaged Code

Unmanaged code executes directly by the operating system and is not controlled by CLR.

- Memory must generally be managed manually.
- Does not automatically provide CLR services.
- Examples: C/C++ programs using native APIs.



Difference

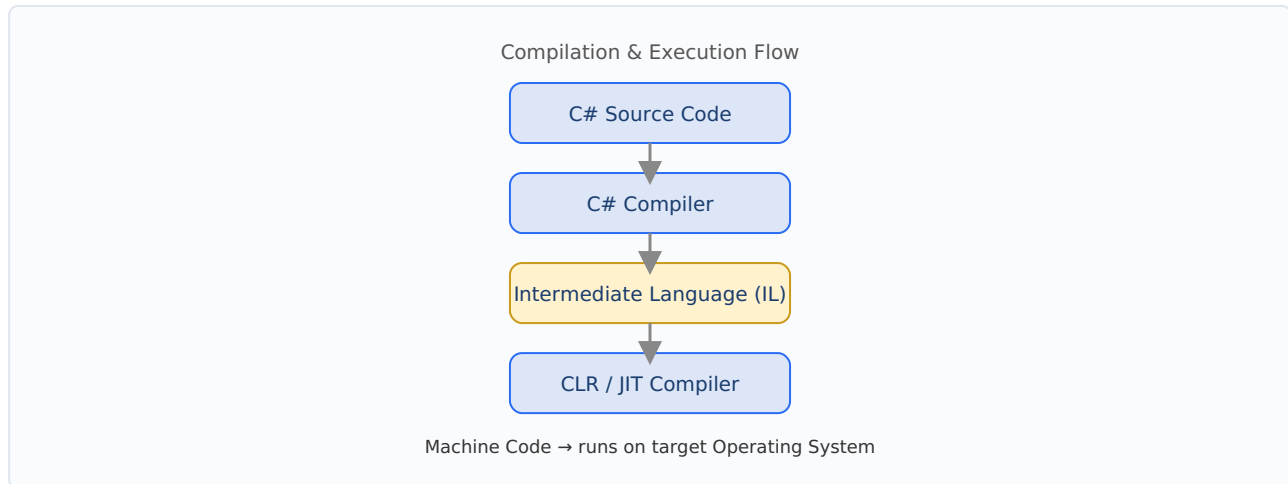
Managed Code	Unmanaged Code
Controlled by CLR	Not controlled by CLR
Automatic memory management	Manual memory management
Provides garbage collection	No CLR garbage collection
More type-safe	Relatively less type-safe
Example: C#	Example: Native C/C++

2 Explain Platform Independence and Language Interoperability in .NET

Platform Independence

Platform independence means that a program can run on different operating systems without changing its source code significantly.

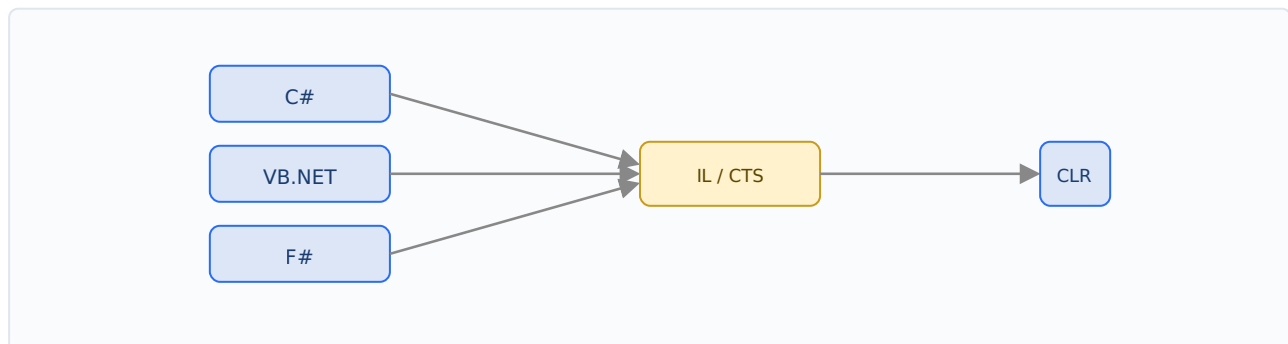
In .NET, source code is first compiled into Intermediate Language (IL). The CLR/JIT compiler converts IL into machine code for the target platform.



Therefore, .NET applications can run on different platforms when a compatible .NET runtime is available.

Language Interoperability

Language interoperability means that different .NET languages can work together because they compile into a common intermediate language and follow the Common Type System (CTS).



A class written in C# can be used by a VB.NET program because both use the .NET type system and runtime.

3 What is a Sealed Class? Explain with Example

A sealed class is a class that cannot be inherited by another class.

The sealed keyword is used to declare it.

```
using System;

sealed class Animal
{
    public void Sound()
    {
        Console.WriteLine("Animal makes sound");
    }
}

// This is not allowed:
// class Dog : Animal { }

class Program
{
    static void Main()
    {
        Animal a = new Animal();
        a.Sound();
    }
}
```

Here, Animal is sealed, so another class cannot inherit from it.

Inheritance blocked by "sealed"

sealed class Animal

class Dog : Animal

Uses

- Prevent inheritance.
- Prevent modification of class behavior through inheritance.
- Useful when a class should have a fixed implementation.

4 Explain Abstract Class and Abstract Method with Example

An abstract class is a class that cannot be instantiated directly. It is generally used as a base class.

An abstract method is a method declared without a body. The derived class must implement it.

```
using System;

abstract class Shape
{
    public abstract void Draw();

    public void Display()
    {
        Console.WriteLine("This is a shape");
    }
}

class Circle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing Circle");
    }
}

class Program
{
    static void Main()
    {
        Circle c = new Circle();
        c.Draw();
        c.Display();
    }
}
```

abstract class Shape

Draw() -- no body
Display() -- implemented

class Circle : Shape

override Draw()

Key points

- Abstract class cannot be instantiated.
- Abstract method has no implementation in the base class.
- Derived class must override the abstract method.
- Abstract classes can contain both abstract and normal methods.

5 Explain Method Overloading and Method Overriding

Method Overloading

Method overloading means having multiple methods with the same name but different parameters in the same class. It is an example of compile-time polymorphism.

```
class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

Here, Add() is overloaded.

Method Overriding

Method overriding occurs when a derived class provides a new implementation of a method defined in the base class using virtual and override.

It is an example of runtime polymorphism.

```
class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("Animal sound");
    }
}

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

Overloading (compile-time)

Add(int a, int b)

Add(int a, int b, int c)

Same name, different parameters

Overriding (runtime)

Animal.Sound() virtual

Dog.Sound() override

Same signature, base to derived

Difference

Overloading	Overriding
Same class generally	Base and derived classes
Different parameters	Same method signature
Compile-time polymorphism	Runtime polymorphism
virtual not required	Uses virtual and override

6 What is Compile-Time Polymorphism? Explain with Example

Compile-time polymorphism is the type of polymorphism where the method to be executed is determined during compilation.

It is mainly achieved through method overloading and operator overloading.

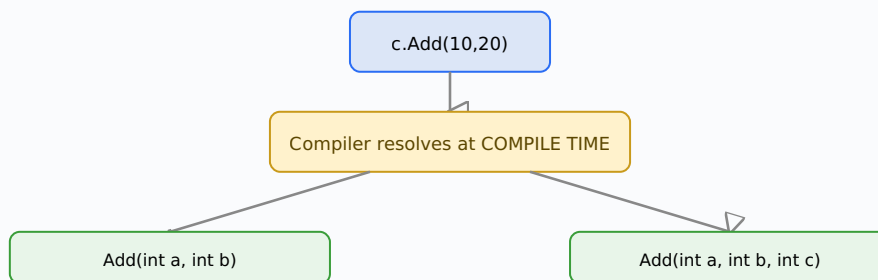
```
class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}

class Program
{
    static void Main()
    {
        Calculator c = new Calculator();

        Console.WriteLine(c.Add(10, 20));
        Console.WriteLine(c.Add(10, 20, 30));
    }
}
```

The compiler determines which Add() method should be called based on the number of arguments.



Output: 30 60

7 What is Runtime Polymorphism / Dynamic Binding? Explain with virtual and override

Runtime polymorphism occurs when the method to be executed is determined at runtime rather than compile time. It is achieved using method overriding, usually with virtual and override.

```
using System;

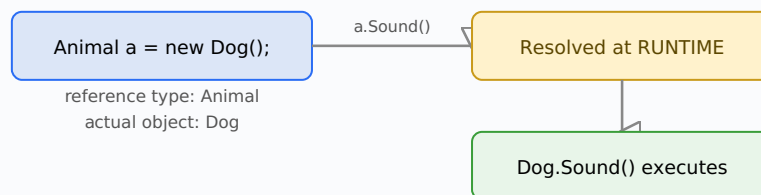
class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("Animal makes sound");
    }
}

class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog barks");
    }
}

class Program
{
    static void Main()
    {
        Animal a = new Dog();
        a.Sound();
    }
}
```

Output: Dog barks

Although a is an Animal reference, it refers to a Dog object. Therefore, the Dog version of Sound() is called at runtime. This is called dynamic binding.



8 Explain Indexers in C# with Example

An indexer allows an object of a class to be accessed like an array using an index.

It is declared using the this keyword.

```
using System;

class Student
{
    private string[] names = new string[3];

    public string this[int index]
    {
        get
        {
            return names[index];
        }
        set
        {
            names[index] = value;
        }
    }
}

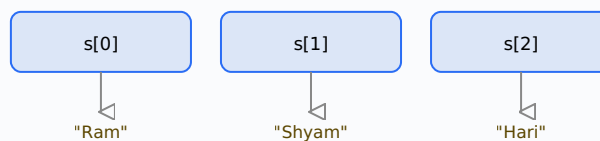
class Program
{
    static void Main()
    {
        Student s = new Student();

        s[0] = "Ram";
        s[1] = "Shyam";
        s[2] = "Hari";

        Console.WriteLine(s[0]);
        Console.WriteLine(s[1]);
    }
}
```

Output: Ram Shyam

s[0] = "Ram" -> accesses object like an array



Advantages

- Provides array-like access to objects.
- Encapsulates internal data.
- Can have get and set accessors.

9 Explain Optional and Named Parameters with Examples

Optional Parameters

An optional parameter has a default value, so the caller does not have to provide it.

```
void Display(string name, int age = 20)
{
    Console.WriteLine(name + " " + age);
}
```

Calling:

```
Display("Ram");
```

uses 20 as the default age.

Named Parameters

Named parameters allow arguments to be passed by specifying the parameter name.

```
void Student(string name, int age)
{
    Console.WriteLine(name + " " + age);
}
```

Calling:

```
Student(age: 21, name: "Ram");
```

The order of arguments does not matter when parameter names are specified.

Optional Parameter

Display("Ram")
age defaults to 20

Named Parameter

Student(age:21, name:"Ram")
order doesn't matter

Benefits

- Improves readability.
- Allows arguments to be passed in different order.
- Optional parameters reduce unnecessary arguments.

10 Explain String Interpolation in C#

String interpolation provides an easy way to insert variables or expressions directly into a string.

It uses the \$ symbol before the string.

```
string name = "Dinesh";  
int age = 22;  
  
Console.WriteLine($"My name is {name} and I am {age} years old.");
```

Output: My name is Dinesh and I am 22 years old.

Expressions can also be used:

```
int a = 10;  
int b = 20;  
  
Console.WriteLine($"Sum = {a + b}");
```

Output: Sum = 30

\$"My name is {name} and I am {age} years old."

name -> "Dinesh"

age -> 22

Advantage: It is more readable and convenient than string concatenation.

11 Explain ref and out Parameters

Both ref and out allow a method to pass arguments by reference.

ref

A variable passed using ref must be initialized before passing it to the method.

```
static void Change(ref int x)
{
    x = 100;
}

static void Main()
{
    int a = 10;

    Change(ref a);

    Console.WriteLine(a);
}
```

Output: 100

out

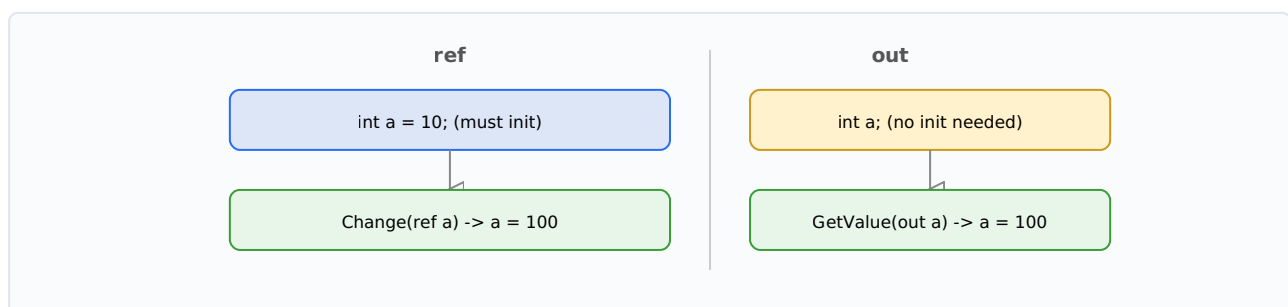
A variable passed using out does not need to be initialized before calling the method. The method must assign a value to it.

```
static void GetValue(out int x)
{
    x = 100;
}

static void Main()
{
    int a;

    GetValue(out a);

    Console.WriteLine(a);
}
```



Difference

ref	out
Must be initialized before method call	Need not be initialized
Method can read and modify it	Method must assign a value
ref keyword required in declaration and call	out keyword required in declaration and call

12 Explain Enumeration (enum) with a C# Program

An enumeration (enum) is a value type used to define a set of named constants.

It improves readability when a variable can have a fixed set of values.

```
using System;

enum Day
{
    Sunday,
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday
}

class Program
{
    static void Main()
    {
        Day today = Day.Monday;

        Console.WriteLine(today);
    }
}
```

Output: Monday

By default, enum values start from 0.

```
Sunday = 0
Monday = 1
Tuesday = 2
...
```

enum Day (values start at 0)

Sunday=0	Monday=1	Tuesday=2	Wednesday=3	Thursday=4	Friday=5	Saturday=6
----------	----------	-----------	-------------	------------	----------	------------

Day today = Day.Monday -> "Monday"

Advantages

- Improves code readability.
- Represents a fixed set of related constants.
- Provides type safety.

13 Explain Arrays and Multidimensional Arrays in C#

An array is a collection of elements of the same data type stored under one variable name.

One-Dimensional Array

```
int[] numbers = { 10, 20, 30, 40, 50 };  
  
Console.WriteLine(numbers[0]);
```

Output: 10

Indexes start from 0.

Multidimensional Array

A multidimensional array contains data in multiple dimensions, such as rows and columns.

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int[,] matrix =  
        {  
            { 1, 2, 3 },  
            { 4, 5, 6 }  
        };  
  
        Console.WriteLine(matrix[0, 1]);  
        Console.WriteLine(matrix[1, 2]);  
    }  
}
```

Output: 2 6

Here:

```
1 2 3  
4 5 6
```

matrix[0,1] refers to 2.

1-D array: numbers[]

10	20	30	40	50
----	----	----	----	----

idx: 0 1 2 3 4

2-D array: matrix[,]

1	2	3
4	5	6

Types of arrays in C#

- One-dimensional array
- Multidimensional array
- Jagged array

14 Explain Events and Event Handling in C#

An event is a mechanism used by a class to notify other classes when something happens. Events are commonly used with delegates.

Basic structure

Event Source
↓
Event
↓
Event Handler
↓
Action performed

```
using System;

class Button
{
    public event EventHandler Click;

    public void Press()
    {
        Click?.Invoke(this, EventArgs.Empty);
    }
}

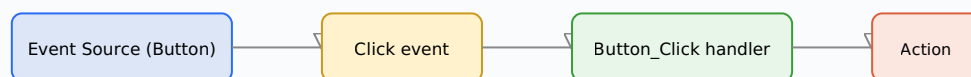
class Program
{
    static void Main()
    {
        Button button = new Button();

        button.Click += Button_Click;

        button.Press();
    }

    static void Button_Click(object sender, EventArgs e)
    {
        Console.WriteLine("Button clicked");
    }
}
```

Output: Button clicked



Here:

- Click is the event.
- Button_Click is the event handler.
- += subscribes the handler to the event.
- Invoke() raises the event.

Examples of events: button click, mouse movement, key press, text change, etc.

15 Short Notes

A. Copy Constructor

A copy constructor is a constructor that creates a new object by copying the values of an existing object of the same class.

```
class Student
{
    public string name;
    public int age;

    public Student(string n, int a)
    {
        name = n;
        age = a;
    }

    public Student(Student s)
    {
        name = s.name;
        age = s.age;
    }
}

class Program
{
    static void Main()
    {
        Student s1 = new Student("Ram", 20);
        Student s2 = new Student(s1);
    }
}
```

Here, s2 receives the values of s1.



B. Indexer

An indexer allows an object to be accessed using an index like an array.

Syntax:

```
public dataType this[int index]
{
    get { ... }
    set { ... }
}
```

```
class Numbers
{
    private int[] n = new int[5];

    public int this[int index]
    {
        get { return n[index]; }
        set { n[index] = value; }
    }
}
```

Usage:


```
Numbers obj = new Numbers();  
  
obj[0] = 10;  
  
Console.WriteLine(obj[0]);
```

C. Lambda Expression

A lambda expression is an anonymous function that provides a short way to write methods or expressions.

It uses the => operator.

Syntax:

```
(parameters) => expression
```

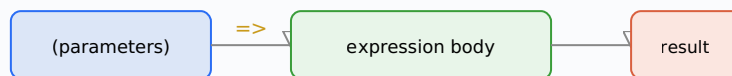
```
Func<int, int> square = x => x * x;  
  
Console.WriteLine(square(5));
```

Output: 25

Another example:

```
Func<int, int, int> add = (a, b) => a + b;  
  
Console.WriteLine(add(10, 20));
```

Output: 30



Advantages

- Short and readable syntax.
- Useful with delegates.
- Widely used with LINQ.
- Can be used for simple anonymous functions.